



Supervision & Observabilité

Concepts fondamentaux et premiers pas

SESSION 1 • 4 H

Au programme de cette séance

4 heures pour comprendre avant de manipuler

1

Introduction à la Supervision

Pourquoi superviser, limites des approches classiques

1h

2

Domaines de surveillance

Performance, disponibilité, niveaux, journalisation

1h

3

Observabilité & 3 piliers

Logs, métriques, traces : la triade moderne

1h

4

TP — Logs basiques

Docker, JSON, jq : diagnostiquer un incident

1h

01

BLOC 1

Introduction à la Supervision

Comprendre l'héritage du monitoring, ses limites, et pourquoi le monde DevOps a besoin d'une nouvelle approche.

Durée : 1 h • Théorie + débat

Superviser, c'est quoi ?

"Surveiller en continu l'état des systèmes informatiques, détecter les anomalies, et déclencher une action lorsque quelque chose ne se passe pas comme prévu."



Observer

Collecter des données sur le système



Détecter

Identifier ce qui sort de la normale



Alerter

Prévenir les bonnes personnes au bon moment

Pourquoi c'est critique ?

9 000 €

le coût moyen d'1 minute de downtime

Ponemon Institute, étude 2020

Ce qu'une panne coûte vraiment

Ventes perdues

Paniers abandonnés, transactions échouées

Confiance abîmée

Clients qui partent, presse négative

SLA non respectés

Pénalités contractuelles, remboursements

Équipe en pompiers

Stress, heures sup, dette technique

Le contexte a changé

Ce qui était surveillable hier ne l'est plus de la même manière

H I E R

- Quelques serveurs physiques
- Architecture monolithique
- Déploiements mensuels
- IPs fixes, infrastructure stable
- 1 équipe Ops centralisée

A U J O U R D ' H U I

- Centaines de containers éphémères
- Microservices distribués
- Déploiements plusieurs fois par jour
- Auto-scaling, infrastructure mouvante
- DevOps : devs responsables en prod



À vous

Que surveilleriez-vous dans une appli web ?

Côté technique

Côté utilisateur

Côté métier

Côté sécurité

La supervision traditionnelle

Une approche éprouvée, mais réactive

Mécanisme de vérification d'états prédéfinis : « si X dépasse Y, lever une alerte ». L'outil dort jusqu'à ce que quelque chose casse.

Outils historiques du marché

Nagios

1999

Le pionnier, ouvert

Zabbix

2001

Open-source, complet

Centreon

2003

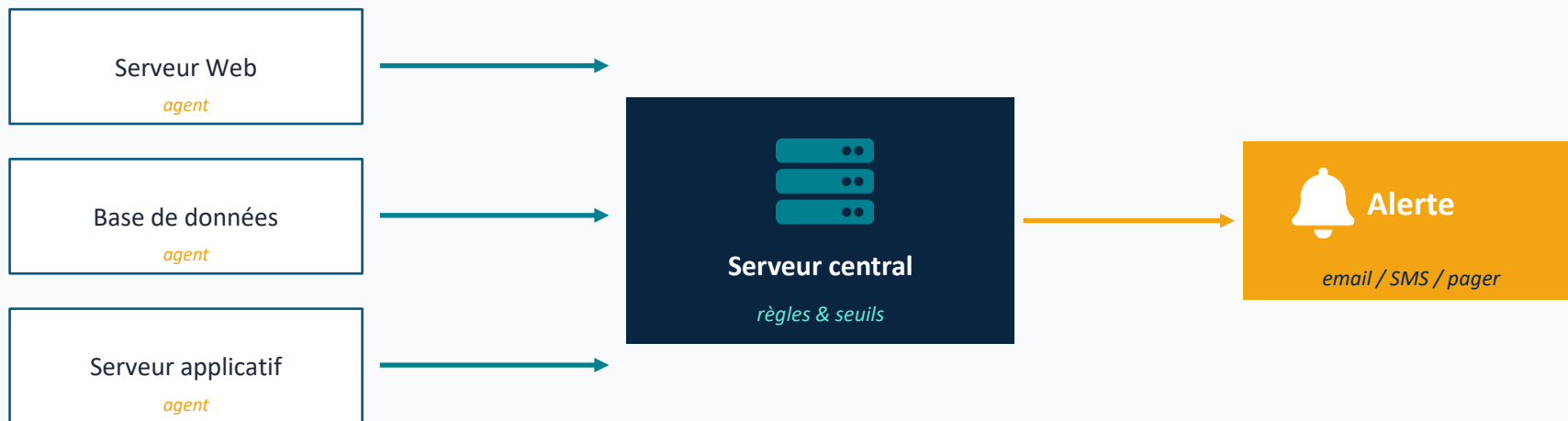
Surcouche Nagios, FR

PRTG

2003

Solution commerciale

Architecture type d'une supervision classique



Boucle : les agents poussent leurs métriques → le serveur évalue les règles → une alerte est levée si un seuil est franchi.

Les limites qui sautent aux yeux

Pourquoi ça ne suffit plus aujourd'hui



Les seuils fixes mentent

« CPU > 80 % » ne dit rien d'un microservice qui tourne à 95 % en régime normal.



Pas de notion de cause

On sait QUE ça casse, jamais POURQUOI. Le diagnostic reste manuel.



Aveugle aux comportements émergents

Les problèmes nouveaux n'ont pas de règle préexistante pour les détecter.



Inadapté à l'infrastructure dynamique

Auto-scaling, containers éphémères : impossible à modéliser à la main.



*If you can't measure it,
you can't improve it.*

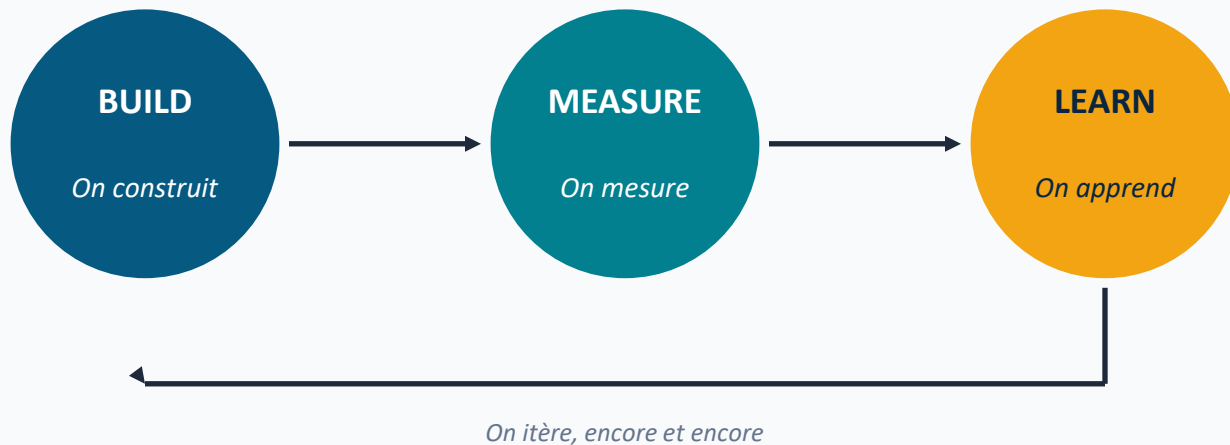
Peter Drucker

Théoricien du management — souvent cité, parfois galvaudé.

L'approche Lean : Build — Measure — Learn

Mesurer n'est pas une fin, c'est le carburant de l'amélioration continue

Sans mesure fiable, on ne peut pas apprendre. Sans apprentissage, on construit à l'aveugle.



Une analogie pour finir le bloc

Du thermostat fixe à la maison qui se comprend elle-même

MAISON D'HIER

Un thermostat, point.

- Un capteur unique : la température
- Une règle fixe : ON si $< 19^{\circ}\text{C}$
- Aucune mémoire, aucun contexte
- On découvre la panne en rentrant

MAISON D'AUJOURD'HUI

Des capteurs partout, qui se parlent.

- Détecteurs partout : CO_2 , humidité, fumée, lumière
- Règles adaptatives selon les habitudes
- Apprend votre routine, anticipe
- Alerte sur le téléphone avant que ça empire

✓ Ce qu'on retient du Bloc 1

La supervision = observer + détecter + alerter, en continu.

Le coût d'une panne est massif : techniques, financiers et humains.

Les outils classiques (Nagios, Zabbix...) ont des limites structurelles : seuils fixes, pas de causalité.

L'infrastructure moderne (cloud, containers, microservices) demande une nouvelle approche.

→ Cap sur le Bloc 2

02

BLOC 2

Domaines de surveillance

*Performance, disponibilité, hiérarchie des niveaux,
journalisation : ce qu'il faut surveiller et pourquoi.*

Durée : 1 h • Théorie + étude de cas

Quatre grands domaines à couvrir

Aucune appli sérieuse ne peut se permettre d'en ignorer un seul



Performance

Latence, débit, ressources



Disponibilité

Uptime, SLA, MTTR, incidents



Niveaux

Système, composants, app, métier



Journalisation

Logs applicatifs, système, audit

Performance — quatre angles à mesurer

Latence

Temps de réponse perçu

EXEMPLES

API : 120 ms p95
Page web : < 2s
P99 paiement : < 500ms

Débit

Volume traité par seconde

EXEMPLES

1 200 req/s
50 K messages/min
800 transactions/s

Ressources

Consommation infra

EXEMPLES

CPU 65 %
RAM 4 GB / 8
I/O disque 200 MB/s

Métier

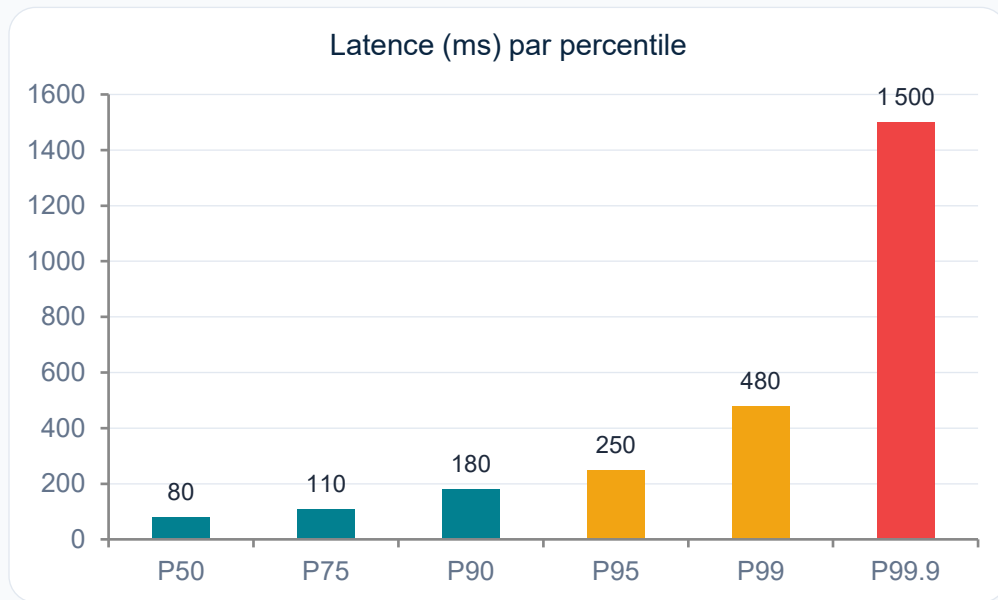
Mesures spécifiques au domaine

EXEMPLES

Temps de checkout
Temps d'indexation
Temps de calcul rapport

La trompeuse moyenne — pensez en percentiles

Pourquoi votre dashboard 'tout va bien' ment à vos utilisateurs



Lire les percentiles

Moyenne

« la plupart de mes users sont à 150 ms »

P95

5 % de vos users attendent ≥ 250 ms

P99

1 % attendent ≥ 480 ms

P99.9

0,1 % à 1,5 s — les outliers tuent l'UX

Performance $\neq$ confort technique. C'est du chiffre d'affaires.

+100 ms

de latence Amazon

= 1 % de ventes en moins

+1 s

de chargement page

= 7 % de conversion perdue

+3 s

d'attente mobile

= 53 % d'abandon de visite

Sources : Amazon (Linden), Akamai (étude e-commerce), Google (Mobile Web Speed)

Disponibilité — les 9 qui coûtent cher

Chaque 9 supplémentaire multiplie quasiment les exigences par 10

SLA	Downtime / an	Downtime / mois	Typique de...
99 %	3,65 jours	7 h 12 min	Site vitrine non critique
99,9 %	8,76 h	43 min	SaaS B2B standard
99,99 %	52 min	4,3 min	Banque en ligne, e-commerce critique
99,999 %	5,26 min	26 s	Téléphonie, infra réseau critique

⚠ Promettre 99,99 % et viser 99,99 % sont deux mondes différents. Chaque 9 coûte exponentiellement plus.

Le vocabulaire des incidents

Trois acronymes à maîtriser pour parler le langage du SRE

MTTR

Mean Time To Recover

Délai moyen entre l'apparition d'un incident et son résolution complète.

● Plus c'est court, mieux c'est

Cible idéale : < 30 min

MTTF

Mean Time To Failure

Durée moyenne pendant laquelle un système fonctionne avant de tomber en panne (composants non réparables).

● Plus c'est long, mieux c'est

Indicateur de fiabilité matérielle

MTBF

Mean Time Between Failures

Durée moyenne entre deux pannes consécutives sur un système réparable.

● Plus c'est long, mieux c'est

Indicateur de robustesse système

SLI, SLO, SLA — l'engagement de service en 3 couches



Quatre niveaux à surveiller

Du métal au métier — chaque étage parle un langage différent



MÉTIER

Taux de conversion, chiffre d'affaires, satisfaction client



APPLICATION

Erreurs HTTP, exceptions, business transactions, logs applicatifs



COMPOSANTS

Pool de connexions DB, hit rate cache, profondeur des queues



SYSTÈME

CPU, RAM, I/O disque, paquets réseau, processus

▲ Plus on monte, plus on parle aux décideurs. ▼ Plus on descend, plus on parle aux ops.

Journalisation — quatre familles de logs

Tous ont leur utilité, aucun ne couvre tout

Applicatifs

Émis par votre code

```
Login failed for user@example.com
```

Système

Émis par l'OS

```
OOM killer triggered on PID 4521
```

Infrastructure

Docker, K8s, cloud

```
Pod evicted: MemoryPressure
```

Audit

Qui a fait quoi

```
Admin user_42 rotated API key X
```

Les logs : la mémoire du système

Sans logs, le diagnostic est de la voyance

*« Les métriques vous disent que quelque chose ne va pas.
Les logs vous disent quoi. »*

Trois rôles essentiels

Reconstituer

Refaire la timeline d'un incident, minute par minute

Comprendre

Passer du symptôme (« lenteur ») à la cause (« requête SQL bloquante »)

Auditer

Tracer qui a accédé à quoi, conformité RGPD/SOC2

Étude de cas — application e-commerce

Pour chaque incident, quel domaine vous aurait sauvé ?

1**Pic d'abandons de panier le vendredi soir***Conversion en chute, latence checkout > 3s***Métier + Performance****2****Le site est down 4 minutes***Health check qui ne répond plus***Disponibilité****3****Un user signale une commande dupliquée***Logs montrent double POST /order***Application + Audit****4****Tentatives de login massives depuis l'étranger***Pic d'auth failed depuis IP suspecte***Sécurité + Audit**

Activité : par groupe de 2-3, identifier les métriques précises à surveiller pour chaque cas (15 min).

✓ Ce qu'on retient du Bloc 2

Quatre domaines : performance, disponibilité, niveaux, journalisation. Aucun n'est facultatif.

Pour la performance, raisonnez en percentiles (P95, P99), pas en moyennes.

SLI $\mapsto$ SLO $\mapsto$ SLA : la chaîne qui transforme une mesure en engagement contractuel.

Logs $\neq$ métriques. Les premières disent QUE, les seconds disent QUOI.

→ Cap sur le Bloc 3

03

BLOC 3

Observabilité & 3 Piliers

*Au-delà de la supervision : comprendre POURQUOI ça casse,
pas seulement QUE ça casse.*

Durée : 1 h • Théorie + quiz + scénario

Qu'est-ce que l'observabilité ?

« Un système est observable si on peut, à partir de ses sorties externes seules, déduire son état interne. »

Rudolf Kálmán, 1960 — théorie du contrôle (avant même l'informatique).

Traduit en langage DevOps

Sans déployer de nouveau code, ni ajouter d'agents, vous pouvez répondre à n'importe quelle question qu'on vous pose sur le comportement de votre système — y compris des questions que vous n'aviez pas anticipées.

Supervision vs Observabilité

Pas un remplacement — un changement de paradigme

Axe	SUPERVISION	OBSERVABILITÉ
Question principale	Quelque chose va-t-il mal ?	Pourquoi et où ça va mal ?
Données	Métriques agrégées, fixes	Données granulaires, riches
Alertes	Sur seuils prédéfinis	Sur patterns détectés
Posture	Réactive — on attend l'alarme	Proactive — on explore en continu
Connues / Inconnues	Couvre les inconnues connues	Couvre les inconnues inconnues

Une analogie qui fait mouche

Thermomètre vs Caméra infrarouge

SUPERVISION = Thermomètre

Donne 1 information : la température. Point.

- « 19,2°C » → ok ou pas ?
- Aucune indication d'où vient le froid
- Pas de notion de fuite, de surface, de gradient
- Réponse : binaire (chaud / froid)

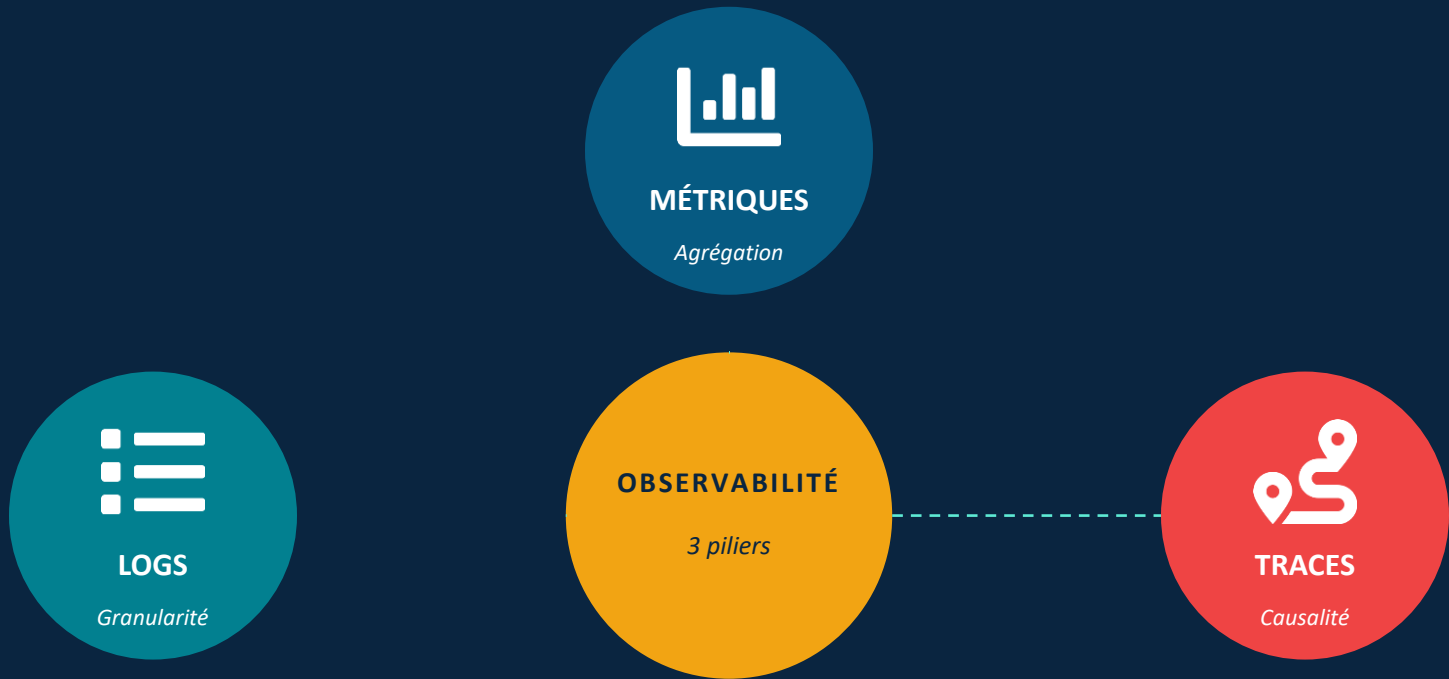
OBSERVABILITÉ = Caméra IR

Donne la carte complète des températures dans le temps.

- Voit OÙ exactement il fait chaud / froid
- Détecte les fuites, ponts thermiques
- Compare l'état actuel à l'état nominal
- Permet des questions ad-hoc : « et là, derrière ce mur ? »

Les 3 piliers de l'observabilité

Aucun n'est suffisant seul. Les trois ensemble = visibilité totale.



Une donnée seule ne raconte rien — c'est la corrélation entre les trois qui produit du sens.

1

LOGS

Événements granulaires et contextuels

Volume

Très élevé*Millions d'événements par jour*

Format

Texte structuré*JSON pour pouvoir être interrogé*

Détail

Maximal*Stacktraces, contexte, IDs*

Cas d'usage

Débogage*Audit, historique précis*

QUESTIONS AUXQUELLES LES LOGS RÉPONDENT

- Quelle erreur exacte s'est produite, avec quels arguments ?
- Quel utilisateur a fait quoi, à quel moment ?
- Quelle séquence d'événements a précédé l'incident ?

À quoi ressemble un log structuré

Un objet JSON par événement, des champs prévisibles

log.json

```
{
  "timestamp": "2025-05-16T10:34:22Z",
  "level": "ERROR",
  "service": "api-gateway",
  "trace_id": "abc123def456",
  "message": "Database connection timeout",
  "context": {
    "user_id": "user_456",
    "endpoint": "/api/checkout",
    "db_pool_active": 50,
    "latency_ms": 5000
  }
}
```

Pourquoi c'est puissant

Filtrable

Par niveau, service, trace_id...

Traçable

trace_id relie tous les logs d'une même requête

Riche

context = données métier embarquées

Aggrégable

Comptage, grouping par n'importe quel champ

2

MÉTRIQUES

Séries temporelles agrégées

Volume

Faible

Échantillons réguliers (15s, 1min...)

Format

Nombre + labels

Très compact, parfait pour le stockage long-terme

Détail

Agrégé

On perd le grain individuel

Cas d'usage

Tendances

Alertes, dashboards, prévisions

QUESTIONS AUXQUELLES LES MÉTRIQUES RÉPONDENT

- Quelle est la tendance du taux d'erreur sur les dernières 24h ?
- Sommes-nous dans notre SLO actuellement ?
- Combien de requêtes par seconde sur l'endpoint X ?

À quoi ressemble une métrique

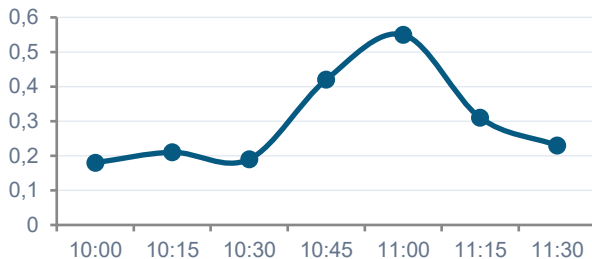
Une valeur, un timestamp, des labels — collectés toutes les N secondes

metrics.promql

```
# Métrique au format Prometheus
http_request_duration_seconds{
  service="api-gateway",
  method="POST",
  endpoint="/api/checkout",
  quantile="0.95"
} 0.234 @ 1747391700

# Exemple de requête
rate(http_requests_total{
  status=~"5.."
}[5m]) > 0.05
```

Latence P95 — dernière heure



Ce qu'on voit ici

Pic de latence vers 10:45-11:00. La métrique signale l'anomalie, mais ne dit pas ce qui s'est passé.

3

TRACES

Le parcours complet d'une requête à travers le système

Volume

Moyen

Sampling pour ne pas exploser le coût

Format

Trace ID + spans

Arbre hiérarchique d'opérations

Détail

Causal

*Qui a appelé qui, dans quel ordre,
combien de temps*

Cas d'usage

Goulets

Détecter où le temps se perd

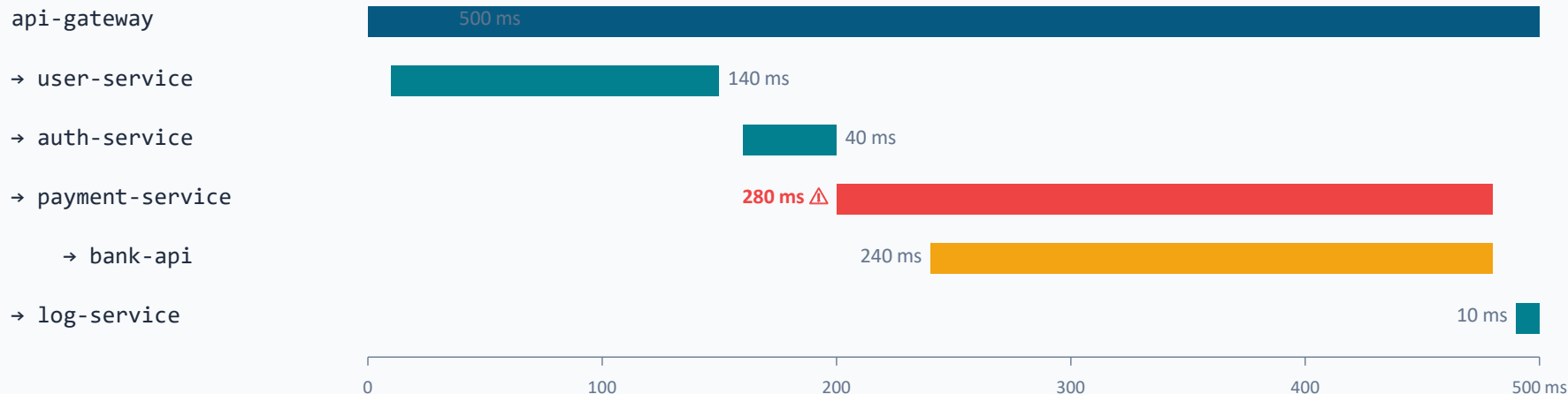
QUESTIONS AUXQUELLES LES TRACES RÉPONDENT

- Pour CETTE requête précise, où le temps a-t-il été passé ?
- Quel service est le goulet d'étranglement dans la chaîne ?
- Quels services en amont/aval sont touchés par une panne ?

À quoi ressemble une trace

Un arbre d'opérations — chaque branche = un service appelé

Trace ID : abc123def456 • Durée totale : 500 ms • /api/checkout



Diagnostic immédiat : payment-service consomme 56% du temps total. Le coupable est identifié en 1 coup d'œil.

Les 3 piliers en action

Incident réel : « la page checkout met 5 secondes »

AVEC SUPERVISION SEULE

Alerte : « P95 latency > 2s »

Mais POURQUOI ? Aucune idée. On commence à ouvrir des terminaux au hasard.

AVEC LES 3 PILIERS

Métriques → on confirme le pic sur payment-service.

Logs → « Database query timeout sur payment-service ».

Traces → 4,8 s sur une requête SQL spécifique.

Cause identifiée en 2 minutes : index manquant sur la table payments.

2 min de diagnostic avec les 3 piliers, contre 30 min sans. ×15 sur le MTTR.

Pourquoi les DevOps en raffolent

Quatre bénéfices concrets et mesurables



MTTR divisé par 5 à 10

Diagnostic en minutes, pas en heures



Prévention

Détection des dégradations subtiles avant l'incident



Compréhension métier

Corrélation directe entre tech et impact utilisateur



Autonomie des devs

Plus besoin d'attendre l'équipe SRE pour debugger en prod

✓ Ce qu'on retient du Bloc 3

Supervision répond « QUE ça casse ». Observabilité répond « POURQUOI et OÙ ça casse ».

Trois piliers : Logs (granularité), Métriques (agrégation), Traces (causalité).

Aucun pilier ne suffit seul. C'est la corrélation entre les trois qui produit du sens.

Le gain typique : $MTTR \div 5$ à 10, plus d'autonomie des équipes de dev.

→ Place au TP (Bloc 4)

Place à la pratique

Bloc 4 — TP Logs basiques

Ce que vous allez faire dans la prochaine heure

- 1 Démarrer un container Docker qui génère des logs applicatifs
- 2 Observer la différence entre logs non structurés et logs JSON
- 3 Écrire vos premières requêtes jq pour interroger des logs
- 4 Diagnostiquer un incident réel à partir des logs collectés

Prérequis à vérifier maintenant : Docker installé, terminal ouvert, jq disponible.